

Melhores Práticas para Integração de SentenceTransformer com PyInstaller em Aplicações Flet Web: Estratégias de Carregamento Assíncrono e Prevenção de Tela Branca

Introdução

O desenvolvimento de aplicações web Flet que utilizam modelos de Machine Learning como SentenceTransformer e sua posterior distribuição via PyInstaller apresenta desafios únicos relacionados ao carregamento inicial, especialmente o problema da "tela branca" durante a inicialização. Este guia aborda as técnicas mais modernas e eficientes para carregamento assíncrono de modelos sem bloquear a interface do usuário.

Configuração do PyInstaller para SentenceTransformer

Dependências Essenciais

Para fazer o bundling correto do SentenceTransformer com PyInstaller, é necessário incluir várias dependências ocultas e metadados específicos^{[1][2]}. O comando PyInstaller deve incluir:

```
pyinstaller --onefile --windowed \  
  --collect-data sentence-transformers \  
  --collect-submodules sentence-transformers \  
  --copy-metadata sentence-transformers \  
  --copy-metadata torch \  
  --copy-metadata transformers \  
  --copy-metadata tqdm \  
  --copy-metadata numpy \  
  --copy-metadata tokenizers \  
  --copy-metadata packaging \  
  --copy-metadata filelock \  
  --collect-all transformers \  
  --hidden-import sentence_transformers \  
  --hidden-import torch \  
  --hidden-import transformers \  
  app.py
```

Arquivo de Especificação (.spec)

Uma abordagem mais robusta é criar um arquivo .spec personalizado que gerencie todas as dependências necessárias^{[3][4]}:

```
# -*- mode: python ; coding: utf-8 -*-  
from PyInstaller.utils.hooks import collect_all, collect_data_files  
import os  
  
# Coleta todas as dependências do transformers  
transformers_datas = collect_data_files('transformers', include_py_files=True)  
transformers_binaries, transformers_hiddenimports = collect_all('transformers')[1:]
```

```

# Coleta dependências do sentence-transformers
sentence_transformers_datas = collect_data_files('sentence_transformers',
include_py_files=True)
sentence_transformers_binaries, sentence_transformers_hiddenimports =
collect_all('sentence_transformers')[1:]

# Coleta dependências do torch
torch_datas = collect_data_files('torch', include_py_files=True)
torch_binaries, torch_hiddenimports = collect_all('torch')[1:]

a = Analysis(
    ['app.py'],
    pathex=['.'],
    binaries=transformers_binaries + sentence_transformers_binaries + torch_binaries,
    datas=transformers_datas + sentence_transformers_datas + torch_datas,
    hiddenimports=[
        'sentence_transformers',
        'sentence_transformers.models',
        'sentence_transformers.cross_encoder',
        'transformers',
        'transformers.models',
        'torch',
        'torch.nn',
        'torch.nn.functional',
        'numpy',
        'tokenizers',
        'tqdm',
    ] + transformers_hiddenimports + sentence_transformers_hiddenimports +
    torch_hiddenimports,
    hookspath=[],
    runtime_hooks=[],
    excludes=[],
    win_no_prefer_redirects=False,
    win_private_assemblies=False,
    cipher=None,
    noarchive=False,
)

pyz = PYZ(a.pure, a.zipped_data, cipher=None)

exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.zipfiles,
    a.datas,
    [],
    name='app',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    upx_exclude=[],
    runtime_tmpdir=None,
    console=False,

```

```
disable_windowed_traceback=False,  
argv_emulation=False,  
target_arch=None,  
codesign_identity=None,  
entitlements_file=None,  
)
```

Estratégia de Threading para Carregamento Assíncrono

Arquitetura Moderna do Flet

O Flet 0.21.0 introduziu melhorias significativas na arquitetura assíncrona, tornando-se um framework "async-first" que permite misturar métodos síncronos e assíncronos na mesma aplicação^{[5][6]}. A estratégia recomendada utiliza:

1. **Métodos síncronos** executados em threads separadas
2. **Métodos assíncronos** executados como Tasks do asyncio
3. **Comunicação thread-safe** via `page.run_task()` e `page.run_thread()`

Implementação do Carregamento Assíncrono

```
import flet as ft  
import asyncio  
import threading  
from sentence_transformers import SentenceTransformer  
import time  
  
class ModelLoader:  
    def __init__(self, page: ft.Page):  
        self.page = page  
        self.model = None  
        self.loading_complete = threading.Event()  
        self.loading_progress = 0  
  
    def load_model_sync(self, model_name: str, progress_callback=None):  
        """Carrega o modelo de forma síncrona em thread separada"""  
        try:  
            # Simula progresso de carregamento  
            steps = [  
                ("Inicializando carregador...", 20),  
                ("Baixando modelo...", 50),  
                ("Carregando pesos...", 80),  
                ("Finalizando configuração...", 100)  
            ]  
  
            for step_name, progress in steps:  
                if progress_callback:  
                    progress_callback(step_name, progress)  
                time.sleep(0.5) # Simula tempo de carregamento  
  
            # Carregamento real do modelo  
            self.model = SentenceTransformer(model_name)  
            self.loading_complete.set()
```

```

        if progress_callback:
            progress_callback("Modelo carregado com sucesso!", 100)

    except Exception as e:
        if progress_callback:
            progress_callback(f"Erro no carregamento: {str(e)}", -1)

    async def load_model_async(self, model_name: str, progress_callback=None):
        """Versão assíncrona do carregamento"""
        try:
            steps = [
                ("Inicializando carregador...", 20),
                ("Baixando modelo...", 50),
                ("Carregando pesos...", 80),
                ("Finalizando configuração...", 100)
            ]

            for step_name, progress in steps:
                if progress_callback:
                    await progress_callback(step_name, progress)
                await asyncio.sleep(0.5) # Simula tempo de carregamento

            # Carregamento real do modelo
            self.model = SentenceTransformer(model_name)

            if progress_callback:
                await progress_callback("Modelo carregado com sucesso!", 100)

        except Exception as e:
            if progress_callback:
                await progress_callback(f"Erro no carregamento: {str(e)}", -1)

class LoadingScreen:
    def __init__(self, page: ft.Page):
        self.page = page
        self.progress_bar = ft.ProgressBar(
            width=400,
            color=ft.colors.BLUE,
            bgcolor=ft.colors.GREY_300,
            value=0
        )
        self.status_text = ft.Text(
            "Inicializando...",
            size=16,
            text_align=ft.TextAlign.CENTER
        )
        self.container = ft.Container(
            content=ft.Column([
                ft.Text("Carregando Modelo", size=24, weight=ft.FontWeight.BOLD),
                self.status_text,
                self.progress_bar,
            ],
            alignment=ft.MainAxisAlignment.CENTER,
            horizontal_alignment=ft.CrossAxisAlignment.CENTER),
            alignment=ft.alignment.center,

```

```

        padding=20
    )

def update_progress(self, status: str, progress: int):
    """Atualiza o progresso da tela de carregamento (thread-safe)"""
    def update_ui():
        self.status_text.value = status
        if progress >= 0:
            self.progress_bar.value = progress / 100
        self.page.update()

    # Executa a atualização na thread principal
    self.page.run_thread(update_ui)

async def update_progress_async(self, status: str, progress: int):
    """Versão assíncrona da atualização de progresso"""
    self.status_text.value = status
    if progress >= 0:
        self.progress_bar.value = progress / 100
    self.page.update()

async def main(page: ft.Page):
    page.title = "Carregamento Assíncrono de Modelo"
    page.vertical_alignment = ft.MainAxisAlignment.CENTER
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

    # Inicializa componentes
    loading_screen = LoadingScreen(page)
    model_loader = ModelLoader(page)

    # Exibe tela de carregamento
    page.add(loading_screen.container)

    # Estratégia 1: Carregamento síncrono em thread separada
    def start_loading_sync():
        model_loader.load_model_sync(
            "all-MiniLM-L6-v2",
            progress_callback=loading_screen.update_progress
        )

    # Notifica conclusão via task assíncrona
    page.run_task(on_loading_complete)

    # Estratégia 2: Carregamento totalmente assíncrono
    async def start_loading_async():
        await model_loader.load_model_async(
            "all-MiniLM-L6-v2",
            progress_callback=loading_screen.update_progress_async
        )
        await on_loading_complete()

    async def on_loading_complete():
        """Callback executado quando o carregamento é concluído"""
        # Remove tela de carregamento
        page.clean()

```

```

# Adiciona interface principal
page.add(ft.Column([
    ft.Text("Modelo carregado com sucesso!", size=20),
    ft.ElevatedButton(
        "Testar Modelo",
        on_click=lambda _: test_model()
    )
], alignment=ft.MainAxisAlignment.CENTER))

def test_model():
    """Testa o modelo carregado"""
    if model_loader.model:
        # Exemplo de uso do modelo
        sentences = ["Hello world", "Flet is awesome"]
        embeddings = model_loader.model.encode(sentences)

        page.add(ft.Text(f"Embeddings shape: {embeddings.shape}"))
        page.update()

# Escolha da estratégia de carregamento
# Opção 1: Thread separada (recomendada para modelos grandes)
threading.Thread(target=start_loading_sync, daemon=True).start()

# Opção 2: Carregamento totalmente assíncrono
# await start_loading_async()

# Execução da aplicação
ft.app(main)

```

Padrões de Notificação Thread-Safe

Comunicação entre Threads

O Flet fornece métodos específicos para comunicação segura entre threads^{[7][8]}:

```

class ThreadSafeNotifier:
    def __init__(self, page: ft.Page):
        self.page = page
        self._callbacks = []

    def add_callback(self, callback):
        """Adiciona callback para notificações"""
        self._callbacks.append(callback)

    def notify_sync(self, data):
        """Notifica de forma síncrona"""
        def update_ui():
            for callback in self._callbacks:
                callback(data)

        self.page.run_thread(update_ui)

    async def notify_async(self, data):
        """Notifica de forma assíncrona"""

```

```

for callback in self._callbacks:
    if asyncio.iscoroutinefunction(callback):
        await callback(data)
    else:
        callback(data)

```

Padrão Producer-Consumer com Queue

```

import queue
import threading

class ModelLoadingQueue:
    def __init__(self, page: ft.Page):
        self.page = page
        self.task_queue = queue.Queue()
        self.result_queue = queue.Queue()
        self.worker_thread = None
        self.is_running = False

    def start_worker(self):
        """Inicia thread worker"""
        self.is_running = True
        self.worker_thread = threading.Thread(target=self._worker, daemon=True)
        self.worker_thread.start()

        # Inicia task para processar resultados
        self.page.run_task(self._process_results)

    def _worker(self):
        """Thread worker que processa tarefas"""
        while self.is_running:
            try:
                task = self.task_queue.get(timeout=1)
                if task is None:
                    break

                # Processa tarefa
                result = self._process_task(task)
                self.result_queue.put(result)

            except queue.Empty:
                continue

    def _process_task(self, task):
        """Processa uma tarefa específica"""
        task_type = task.get('type')

        if task_type == 'load_model':
            model_name = task.get('model_name')
            model = SentenceTransformer(model_name)
            return {'type': 'model_loaded', 'model': model}

        return {'type': 'unknown_task'}

    async def _process_results(self):

```

```

        """Processa resultados de forma assíncrona"""
        while self.is_running:
            try:
                result = self.result_queue.get_nowait()
                await self._handle_result(result)
            except queue.Empty:
                await asyncio.sleep(0.1)

    async def _handle_result(self, result):
        """Manipula resultado na thread principal"""
        result_type = result.get('type')

        if result_type == 'model_loaded':
            self.page.add(ft.Text("Modelo carregado com sucesso!"))
            self.page.update()

    def load_model(self, model_name: str):
        """Adiciona tarefa de carregamento à queue"""
        task = {'type': 'load_model', 'model_name': model_name}
        self.task_queue.put(task)

    def stop(self):
        """Para o worker thread"""
        self.is_running = False
        self.task_queue.put(None)
        if self.worker_thread:
            self.worker_thread.join()

```

Otimizações para Prevenção de Tela Branca

Carregamento Progressivo

```

class ProgressiveModelLoader:
    def __init__(self, page: ft.Page):
        self.page = page
        self.components = []

    def add_loading_component(self, component):
        """Adiciona componente de carregamento"""
        self.components.append(component)
        self.page.add(component)
        self.page.update()

    async def load_with_ui_updates(self, model_name: str):
        """Carrega modelo com atualizações contínuas da UI"""

        # Fase 1: Preparação
        prep_indicator = ft.ProgressRing(width=50, height=50)
        prep_text = ft.Text("Preparando ambiente...")
        self.add_loading_component(ft.Row([prep_indicator, prep_text]))
        await asyncio.sleep(0.1)  # Permite atualização da UI

        # Fase 2: Download
        download_bar = ft.ProgressBar(width=300, value=0)

```



```

download_text = ft.Text("Baixando modelo...")
self.add_loading_component(ft.Column([download_text, download_bar]))

# Simula progresso de download
for i in range(0, 101, 10):
    download_bar.value = i / 100
    download_text.value = f"Baixando modelo... {i}%"
    self.page.update()
    await asyncio.sleep(0.1)

# Fase 3: Carregamento
load_indicator = ft.ProgressRing(width=50, height=50)
load_text = ft.Text("Carregando modelo na memória...")
self.add_loading_component(ft.Row([load_indicator, load_text]))

# Carregamento real do modelo
model = SentenceTransformer(model_name)

# Fase 4: Finalização
self.page.clean()
self.page.add(ft.Text("Modelo carregado com sucesso!", size=20))
self.page.update()

return model

```

Implementação de Splash Screen

```

class SplashScreen:
    def __init__(self, page: ft.Page):
        self.page = page
        self.splash_container = None

    def show(self):
        """Exibe splash screen"""
        logo = ft.Icon(
            ft.icons.PSYCHOLOGY,
            size=100,
            color=ft.colors.BLUE
        )

        title = ft.Text(
            "AI Assistant",
            size=32,
            weight=ft.FontWeight.BOLD,
            color=ft.colors.BLUE
        )

        loading_ring = ft.ProgressRing(
            width=50,
            height=50,
            color=ft.colors.BLUE
        )

        loading_text = ft.Text(
            "Carregando modelo de IA...",

```

```

        size=16,
        color=ft.colors.GREY_700
    )

    self.splash_container = ft.Container(
        content=ft.Column([
            logo,
            title,
            ft.Container(height=30),
            loading_ring,
            loading_text
        ],
        alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER),
        bgcolor=ft.colors.WHITE,
        padding=40,
        alignment=ft.alignment.center
    )

    self.page.add(self.splash_container)
    self.page.update()

def hide(self):
    """Oculta splash screen"""
    if self.splash_container:
        self.page.remove(self.splash_container)
        self.page.update()

```

Exemplo Completo de Implementação

```

import flet as ft
import asyncio
import threading
import time
from sentence_transformers import SentenceTransformer
from typing import Optional, Callable

class ModelManager:
    def __init__(self):
        self.model: Optional[SentenceTransformer] = None
        self.is_loading = False
        self.loading_progress = 0

    async def load_model_async(self,
                               model_name: str,
                               progress_callback: Optional[Callable] = None) -> bool:
        """Carrega modelo de forma assíncrona"""
        if self.is_loading:
            return False

        self.is_loading = True

        try:
            # Simula fases de carregamento
            phases = [

```

```

        ("Inicializando...", 10),
        ("Validando configuração...", 25),
        ("Baixando modelo...", 60),
        ("Carregando pesos...", 85),
        ("Finalizando...", 100)
    ]

    for phase_name, progress in phases:
        if progress_callback:
            await progress_callback(phase_name, progress)

        # Simula tempo de processamento
        await asyncio.sleep(0.5)

    # Carregamento real do modelo
    self.model = SentenceTransformer(model_name)

    if progress_callback:
        await progress_callback("Modelo carregado!", 100)

    return True

except Exception as e:
    if progress_callback:
        await progress_callback(f"Erro: {str(e)}", -1)
    return False

finally:
    self.is_loading = False

def encode_sentences(self, sentences: list) -> Optional[list]:
    """Codifica sentenças usando o modelo"""
    if not self.model:
        return None

    return self.model.encode(sentences).tolist()

class AppUI:
    def __init__(self, page: ft.Page):
        self.page = page
        self.model_manager = ModelManager()
        self.setup_page()

    def setup_page(self):
        """Configura a página principal"""
        self.page.title = "SentenceTransformer App"
        self.page.theme_mode = ft.ThemeMode.LIGHT
        self.page.padding = 20

    async def show_loading_screen(self, model_name: str):
        """Exibe tela de carregamento"""
        self.page.clean()

        # Elementos da tela de carregamento
        title = ft.Text(
            "Carregando Modelo de IA",

```

```

        size=24,
        weight=ft.FontWeight.BOLD,
        text_align=ft.TextAlign.CENTER
    )

    progress_bar = ft.ProgressBar(
        width=400,
        height=10,
        bgcolor=ft.colors.GREY_300,
        color=ft.colors.BLUE,
        value=0
    )

    status_text = ft.Text(
        "Inicializando...",
        size=14,
        text_align=ft.TextAlign.CENTER,
        color=ft.colors.GREY_700
    )

    loading_container = ft.Container(
        content=ft.Column([
            ft.Icon(ft.icons.PSYCHOLOGY, size=80, color=ft.colors.BLUE),
            title,
            ft.Container(height=20),
            progress_bar,
            ft.Container(height=10),
            status_text
        ]),
        alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER),
        alignment=ft.alignment.center,
        padding=40
    )

    self.page.add(loading_container)
    self.page.update()

    # Callback para atualização de progresso
    async def update_progress(status: str, progress: int):
        status_text.value = status
        if progress >= 0:
            progress_bar.value = progress / 100
            self.page.update()

    # Inicia carregamento do modelo
    success = await self.model_manager.load_model_async(
        model_name,
        update_progress
    )

    if success:
        await asyncio.sleep(1) # Mostra sucesso por um momento
        self.show_main_interface()
    else:
        self.show_error_screen()

```

```

def show_main_interface(self):
    """Exibe interface principal após carregamento"""
    self.page.clean()

    title = ft.Text(
        "SentenceTransformer Carregado!",
        size=24,
        weight=ft.FontWeight.BOLD
    )

    input_field = ft.TextField(
        label="Digite uma frase para codificar",
        multiline=True,
        min_lines=2,
        max_lines=5,
        width=500
    )

    result_text = ft.Text(
        "Resultado aparecerá aqui...",
        size=12,
        color=ft.colors.GREY_600
    )

    def encode_text(e):
        if input_field.value:
            sentences = [input_field.value]
            embeddings = self.model_manager.encode_sentences(sentences)

            if embeddings:
                result_text.value = f"Embedding shape: {len(embeddings[0])}
dimensões"

                result_text.color = ft.colors.GREEN
            else:
                result_text.value = "Erro ao codificar texto"
                result_text.color = ft.colors.RED

            self.page.update()

    encode_button = ft.ElevatedButton(
        "Codificar Texto",
        on_click=encode_text,
        width=200
    )

    main_container = ft.Container(
        content=ft.Column([
            title,
            ft.Container(height=20),
            input_field,
            ft.Container(height=10),
            encode_button,
            ft.Container(height=20),
            result_text
        ]),

```

```

        alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER),
        padding=40
    )

    self.page.add(main_container)
    self.page.update()

def show_error_screen(self):
    """Exibe tela de erro"""
    self.page.clean()

    error_container = ft.Container(
        content=ft.Column([
            ft.Icon(ft.icons.ERROR, size=80, color=ft.colors.RED),
            ft.Text(
                "Erro ao carregar modelo",
                size=24,
                weight=ft.FontWeight.BOLD,
                color=ft.colors.RED
            ),
            ft.Text(
                "Verifique sua conexão e tente novamente",
                size=14,
                color=ft.colors.GREY_700
            ),
            ft.ElevatedButton(
                "Tentar Novamente",
                on_click=lambda _: self.restart_app()
            )
        ]),
        alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER),
        alignment=ft.alignment.center,
        padding=40
    )

    self.page.add(error_container)
    self.page.update()

    async def restart_app(self):
        """Reinicia a aplicação"""
        await self.show_loading_screen("all-MiniLM-L6-v2")

async def main(page: ft.Page):
    """Função principal da aplicação"""
    app = AppUI(page)
    await app.show_loading_screen("all-MiniLM-L6-v2")

# Execução da aplicação
if __name__ == "__main__":
    ft.app(main)

```

Considerações de Performance

Threading vs Asyncio

Para aplicações Flet que carregam modelos de ML, a escolha entre threading e asyncio depende de vários fatores^{[9][10]}:

- **Threading:** Melhor para operações CPU-intensivas como carregamento de modelos
- **Asyncio:** Ideal para operações I/O-bound como download de modelos
- **Híbrido:** Combinação de ambos para máxima eficiência

Otimizações Específicas

1. **Pré-carregamento:** Carregar modelos menores primeiro
2. **Cache Inteligente:** Armazenar modelos em cache local
3. **Carregamento Lazy:** Carregar componentes apenas quando necessário
4. **Paralelização:** Usar múltiplas threads para diferentes componentes

Conclusão

A implementação bem-sucedida de carregamento assíncrono de modelos SentenceTransformer em aplicações Flet requer uma abordagem cuidadosa que combina:

1. **Configuração adequada do PyInstaller** com todas as dependências necessárias^{[11][12]}
2. **Estratégia de threading híbrida** aproveitando as capacidades async-first do Flet^{[5][6]}
3. **Padrões de comunicação thread-safe** para atualizações da UI^{[7][13]}
4. **Interface de usuário responsiva** com feedback visual contínuo^{[14][15]}

Esta abordagem garante uma experiência de usuário fluida, eliminando o problema da tela branca e fornecendo feedback visual durante todo o processo de carregamento do modelo.

*
**